Aluno: Marcos Mauricio da Cunha de Morais

Tema 1: Detector de EPIs com Controle de Porta

Objetivo

A ideia é montar um sistema que confira se a pessoa está usando os EPIs exigidos antes de liberar uma porta. Se algo falhar, seja a rede, o servidor ou o próprio dispositivo, a porta deve continuar trancada. È pretendido também medir com que frequência o sistema deixa passar alguém sem o equipamento (falso negativo), quanto tempo ele leva para responder e como funciona a comunicação em rede entre as partes.

Métodos

Um sensor de distância percebe que alguém chegou e aciona o ESP32-CAM, que tira algumas fotos em sequência e as envia por HTTP POST (TCP) para uma página PHP no notebook. Lá, o OpenCV e um modelo YOLO, treinado com cerca de 50 fotos de cada EPI e com data augmentation, analisam as imagens. A pessoa só é liberada se os EPIs cadastrados pelo administrador aparecerem na maioria das fotos. O resultado é guardado no MySQL e devolvido ao ESP32, que aciona o relé. O ESP32 também manda sinais de hesrtbeat por UDP, para o servidor saber se ele continua ligado. A porta fica trancada por padrão, então queda de energia, falta de resposta ou travamento do dispositivo a mantêm fechada. Na avaliação, a proposta é olhar o recall por classe, o falso negativo e a latência, e comparar HTTP com UDP.

Aplicativos e ferramentas utilizados

YOLO e OpenCV (em Python), Apache e PHP, MySQL, Arduino IDE, LabelImg e Wireshark.

Material físico utilizado

ESP32-CAM, sensor de distância (HC-SR04 ou VL53L0X), módulo relé, trava elétrica que fica trancada sem energia com fonte de 12 V, notebook com Intel i5-7200U como servidor, roteador ou switch para a rede local e os EPIs para montar o conjunto de imagens.

Tema 2: Triagem de Frutas e Grãos com Ejeção Simulada

Objetivo

A proposta é criar um sistema que reconheça frutas e grãos passando em uma esteira e ejete os itens fora do padrão com um braço ou pistão simulado. Pretende-se medir quantos itens defeituosos acabam passando (falso negativo), quanto tempo o sistema leva para reagir e como a comunicação em rede entre câmera, servidor e atuador se comporta.

Métodos

No Gazebo, uma câmera simulada observa a esteira, que se move em velocidade constante e conhecida (a esteira é só o cenário, e o controle de velocidade fica fora do escopo). Os itens são modelados com fotos reais de frutas e grãos. Os quadros passam pelo ROS 2 e são enviados por HTTP POST (TCP) a uma página PHP no notebook, onde o OpenCV e um modelo YOLO, treinado com cerca de 50 fotos de cada classe e com data augmentation, fazem a análise. Quando um item é considerado fora do padrão, o comando de ejeção vai ao atuador simulado no momento calculado pela velocidade da esteira. Os resultados são guardados no MySQL, e heartbeats por UDP indicam se o módulo está ativo. Na avaliação, a ideia é observar o recall por classe, o falso negativo, a latência e o erro de temporização da ejeção, além de comparar HTTP com UDP.

Aplicativos e ferramentas utilizados

YOLO e OpenCV (em Python), Gazebo e ROS 2, Apache e PHP, MySQL, LabelImg e Wireshark.

Material físico utilizado

Notebook com Intel i5-7200U como servidor e ambiente de simulação, roteador ou switch para a rede local, e as frutas e grãos fotografados para o conjunto de imagens e as texturas da simulação.

Tema 3: J.E.I (Just Enough Items): Assistente de Manutenção por Reconhecimento de Máquinas e Peças

Objetivo

O nome vem do mod “Just Enough Items”, do Minecraft, que mostra a receita e os usos de cada item, e foi essa funcionalidade que serviu de inspiração. Aqui, a ideia é levar essa consulta rápida para o mundo real: o sistema identifica uma máquina ou peça por câmera, seja por um QR code cadastrado pelo administrador ou pela aparência, e mostra o que estiver registrado sobre ela, como manutenção, substituição, outros usos possíveis e projetos para fabricá-la. Pretende-se comparar a taxa de acerto e a latência dos dois modos de identificação, além de documentar a comunicação em rede entre câmera, servidor e software.

Métodos

Uma câmera captura a imagem e a envia por HTTP POST (TCP) a uma página PHP no notebook. Se houver QR code, o OpenCV o lê e consulta o item no MySQL. Se não houver, a imagem passa por um modelo YOLO treinado com cerca de 50 fotos de cada item, com data augmentatipn. Nos dois casos, um software em Python mostra o item identificado e pede a confirmação do usuário antes de exibir os procedimentos, com um aviso de que a identificação feita por IA pode errar e deve ser conferida. Quando a confiança é baixa, o sistema avisa que não reconheceu o item e oferece uma busca manual. Ele apenas mostra o que está cadastrado no banco, sem criar procedimentos novos. Heartbeats por UDP indicam se a câmera está ativa. Na avaliação, a proposta é comparar acerto e latência entre QR code e reconhecimento visual, e também HTTP com UDP.

Aplicativos e ferramentas utilizados

YOLO e OpenCV em Python, Tkinter para a interface, Apache e PHP, MySQL, LabelImg e Wireshark.

Material físico utilizado

Câmera (ESP32-CAM ou webcam do notebook), notebook com Intel i5-7200U como servidor, roteador ou switch para a rede local, etiquetas com QR code e as máquinas ou peças usadas para montar o conjunto de imagens.

Tema 4: Estimativa da Maturação de Bananas por Sensor Multiespectral e Câmera RGB

Objetivo

A proposta é avaliar se um sensor multiespectral de baixo custo (AS7265x), uma câmera comum ou os dois combinados conseguem estimar melhor a maturação da banana, usando o teor de açúcar medido por refratômetro (°Brix) como referência. É pretendido comparar o erro de estimativa dos três casos e documentar como as medições chegam do dispositivo embarcado ao servidor.

Métodos

Cada banana é colocada em uma caixa escura portátil, com posições fixas e um cartão de referência de cor. O ESP32-CAM lê o sensor AS7265x e tira uma foto com exposição e balanço de branco fixos. As 18 medidas do sensor e a imagem são enviadas por HTTP POST (TCP) a uma página PHP no notebook, e o OpenCV extrai valores de cor da imagem. Após, se mede o Brix com o refratômetro, usando bananas do mesmo lote, já que a medida exige espremer a fruta. Os dados vão para o MySQL, e heartbeats por UDP indicam se o dispositivo está ativo. Em Python, a ideia é treinar modelos de regressão (PLS ou regressão linear) usando só o sensor, só a cor e os dois juntos, com validação cruzada. A avaliação deve olhar o R^2 e o erro médio em °Brix, além da latência e da comparação entre HTTP e UDP.

Aplicativos e ferramentas utilizados

Python com OpenCV e scikit-learn, Arduino IDE, Apache e PHP, MySQL e Wireshark.

Material físico utilizado

ESP32-CAM, placa com o sensor AS7265x, caixa escura portátil com cartão de referência de cor, power bank, refratômetro de Brix, bananas em diferentes estágios de maturação, notebook com Intel i5-7200U como servidor e um roteador pequeno (ou hotspot do celular) para a rede local.

Tema 5: Sistema de Alerta de Ejeção com Medição de Temporização

Objetivo

A ideia é medir, em hardware real, quanto tempo passa entre a decisão de ejetar um item, tomada na simulação do Tema 2, e o acionamento de um alerta físico, e o quanto esse tempo varia de uma vez para outra (jitter). Também se pretende observar a taxa de erro de ejeção e entender quanto do atraso vem da rede, do processamento e do dispositivo embarcado.

Métodos

Quando a simulação do Tema 2 decide ejetar um item, o notebook envia o comando ao ESP32 por HTTP (TCP) ou UDP. O ESP32 gera um pulso em um pino assim que recebe o comando e acende um LED de alerta em outro pino. Um analisador lógico grava os dois sinais, e o PulseView ajuda a calcular o atraso entre eles em centenas de ejeções. O jitter é a variação desse atraso, e a taxa de erro é a fração de ejeções fora de uma janela de tempo definida pela velocidade constante da esteira. A proposta é comparar HTTP com UDP e o ESP32 com e sem carga de rede, guardando os dados no MySQL. Vale lembrar que o analisador enxerga o sinal elétrico dentro do ESP32 e que a simulação roda no notebook, então a velocidade dela também será anotada.

Aplicativos e ferramentas utilizados

PulseView (sigrok), Python para as estatísticas (média, desvio padrão, percentis), Gazebo e ROS 2 do Tema 2, Arduino IDE, Apache e PHP, MySQL e Wireshark.

Material físico utilizado

ESP32 comum (placa de desenvolvimento), LED com resistor, analisador lógico USB de 8 canais e 24 MHz, jumpers ou protoboard, notebook com Intel i5-7200U como servidor e ambiente de simulação e roteador pequeno para a rede local.